

XAMS Slow Control — Replacement Plan

Inventory, recovered configuration, architecture, and migration sequence · 16 September 2026

1. Situation

XAMS slow control runs as a LabVIEW project (`Slow_Control_2_17_User.vi` , revision 393) that has become unmaintainable. The problem is the software, not the instrumentation: the hardware is a small, clean set of instruments.

The underlying cause is LabVIEW's *form* rather than its capability. VIs are binary files. They cannot be diffed, merged or reviewed; version control degrades to filename suffixes; and a block diagram resists incremental change once it exceeds a screen or two. Knowledge of the system therefore lives in one person's head instead of in a readable artefact. Any replacement must fix that property first.

Conclusion

Given the actual inventory — four instruments, roughly thirty connected channels, one rack — this is a **small** rewrite. Build a **Python service stack**: device drivers publishing to MQTT, PostgreSQL for history, Grafana for plots, and a small alarm service for notifications.

The system stays on **Windows**, on the existing control PC (§7). EPICS was considered seriously and is **not** recommended at this scale (§9). It remains buildable if circumstances change.

Most of the configuration knowledge has now been recovered from the MAX report and the LabVIEW documentation export (§3). What remains missing is listed in §4 and is obtainable in an afternoon.

2. Hardware inventory

Device	Link	Channels	Role	Python support
NI cDAQ-9174 cDAQ1 SN 020C5E1C	USB 2.0	40 available 20 connected (see §3)	Read only	Vendor <code>nidaqmx</code> package. Tier 1.
2x CAEN DT1470ET COM8, COM5	USB (virtual COM)	8 HV	Actuation	CAEN ASCII protocol over <code>pyserial</code> . Tier 2. No vendor library.
Lake Shore 335 COM6	USB (virtual COM)	2 sensor in 2 heater out	Actuation	Official <code>lakeshore</code> package. Tier 1.
UPS model TBD	TBD	status	Read only	Depends on model. <code>NUT</code> or vendor protocol.

Control host: Fujitsu CELSIUS J5010, Xeon W-1270, 32 GB RAM, 953 GB disk, Windows 10 Enterprise LTSC, hostname `NIKHEFN-01AEIIN` . Ample for the replacement stack, and **retained as-is** — see §7.

No device requires protocol reverse-engineering. Two observations:

- **Everything is on the USB hub** — cDAQ, both CAEN supplies, and the Lake Shore. Data rates are trivial, so bandwidth is a non-issue, but device identification is not (see below). The DT1470ET also has an Ethernet port, which remains available as a fallback should USB prove unreliable.
- **The cDAQ chassis cannot actuate anything** — no digital or analog output modules. It is a pure monitoring device, so its service carries no control path and no interlock responsibility. This makes it the safest component to build first.

Two identical HV supplies on USB — identification

Both CAEN units are the same model. Confusing them means applying the wrong voltage to the wrong electrode, so identification must not rest on enumeration order. Use both layers.

1. Resolve the port by hardware ID, not by COM number. Windows assigns COM numbers per device instance and they can change. Match on the USB hardware ID in Python, which behaves identically on Windows and Linux:

```
from serial.tools import list_ports
for p in list_ports.comports():
    print(p.device, p.hwid)          # e.g. COM8  USB VID:PID=0403:6001 SER=AB12CD34
```

If the units expose unique serial numbers, match on `SER=`. If both report the same value — some FTDI chips ship without unique serials — match on the USB location instead, which ties correctness to a hub socket and therefore requires a physical label on the hub.

2. Verify identity in software on every connect. This is the layer that actually protects the detector, because it does not depend on cabling discipline:

```
$BD:00,CMD:MON,PAR:BDSNUM    → serial number
$BD:00,CMD:MON,PAR:BDNAME    → model
```

Record the expected serial number in the channel configuration. On connect, query it; **on mismatch the service refuses to start and raises an alarm**. It never falls back to the likely candidate. Set distinct board addresses (`BD`) on the two front panels as well, so the protocol itself disambiguates.

3. Configuration recovered from the existing system

Extracted from the NI-MAX configuration report and the LabVIEW documentation export. NI-MAX *Data Neighborhood* is empty, so all DAQmx configuration is built in the block diagram and read off the front panel.

3.1 Chassis and module aliases

LabVIEW addresses the modules by **alias**, not by `cDAQ1ModN`. Keep the aliases: they are more readable and they survive re-slotting.

Slot	Model	Alias	Serial number	Channels
1	NI 9207	9207	020DFD57	8× voltage <code>ai0:7</code> , 8× current <code>ai8:15</code>
2	NI 9216	9216_1	020F64D1	8× RTD <code>ai0:7</code>
3	NI 9216	9216_2	020F64D0	8× RTD <code>ai0:7</code>
4	NI 9226	9226	02159CBB	8× RTD <code>ai0:7</code>

The front panel labels `9216_1` as "slot 1"; MAX reports it in slot 2. The MAX report is authoritative. Do not carry the mislabel over.

3.2 RTD configuration — confirmed

Module	R_0	RTD type	Wiring	Timing
NI 9216 (both)	100,0 Ω	Pt3851	3-wire	High Speed
NI 9226	1000,0 Ω	Pt3750	3-wire	High Speed

No custom Callendar–Van Dusen coefficients are in use (A, B, C all zero), so DAQmx's built-in scaling applies directly: `add_ai_rtd_chan` with `RTDType.PT_3851 / PT_3750`, `ResistanceConfiguration.THREE_WIRE`, and the matching `r_0`.

3.3 RTD channel map

Supplied by the lab, September 2026. **Fourteen RTDs are in use, not twenty-four.**

Module	Type	Channel	Tag	Notes
9226	PT1000	ai0 ... ai6	TT201 ... TT207	seven consecutive
9226	PT1000	ai7	—	not connected
9216_1	PT100	ai0	TT301	
9216_1	PT100	ai1	TT302	
9216_1	PT100	ai2	TT103	
9216_1	PT100	ai3	TT104	
9216_1	PT100	ai4	TTAMB	ambient
9216_1	PT100	ai5	TT303	
9216_1	PT100	ai6	TT304	
9216_1	PT100	ai7	—	not connected
9216_2	PT100	ai0 ... ai7	—	module entirely unconnected

The whole of 9216_2 is free: eight spare PT100 inputs already wired into the chassis, so the system has room to grow without buying hardware. The replacement should not create a DAQmx task for it, but should list its channels in `channels.yaml` as disabled, so the channel map stays complete and a future reader does not have to wonder whether the gap is an oversight.

The numbering groups by subsystem — P101 – P104 and TT103 – TT104 share the 1xx series, TT201 – TT207 the 2xx, TT301 – TT304 the 3xx. What each series denotes, and where each sensor physically sits, is still only lab knowledge.

3.4 NI 9207 voltage channels — names and scaling

The applied value is $(\text{raw} - \text{offset}) \times \text{multiplier}$.

Channel	Name	Offset	Multiplier	Role
9207/ai0	P101	1	25	
9207/ai1	P102	0	1	
9207/ai2	P103	0	1	
9207/ai3	P104	0	1	
9207/ai4	v4	0	1	generic name — likely unused
9207/ai5	Pmain	0	0,714	detector pressure input
9207/ai6	v6	0	1	generic name — likely unused
9207/ai7	FM101	0	6	flow meter input

Current channels ai8:15 are named i0 – i7 with offset 0 and multiplier 1 throughout, which suggests they are unused. Confirm before dropping them.

3.5 CAEN HV channels

Eight channels across the two supplies: PMT top, PMT bot, TS, BS, Cathode, Gate, Anode, NaI. The anode is on **CAEN 2, channel 3**. The existing LabVIEW driver is CAEN's serial PSM instrument driver, which confirms the ASCII-

over-serial route for the replacement.

Two HV-specific behaviours exist in the current code and need explicit decisions: `anode_timing.vi` (anode timers) and `DAISY_polarity_signs.vi` (polarity handling).

3.6 Lake Shore 335

COM6. PID settings $P = 100$, $I = 20$, $D = 0$. A front-panel switch "Shut off heater if Alarm is active" is present — see the safety note in §6.

3.7 Loop timing, averaging, notification

- Cycle time 2 s.
- Moving averages: flow over $N = 2$ samples, detector pressure over $N = 20$ samples.
- Integrated flow via `Average_flow_x_time.vi`.
- Alarm notification: SMS (already a Python script, `send_sms.vi`), email (`custom_email_program.vi`), and a 1000 Hz audible alarm. SMS repeats every N minutes to a list of phone numbers.
- A `KILL_VOLTAGE` button and an alarm master switch are on the front panel.

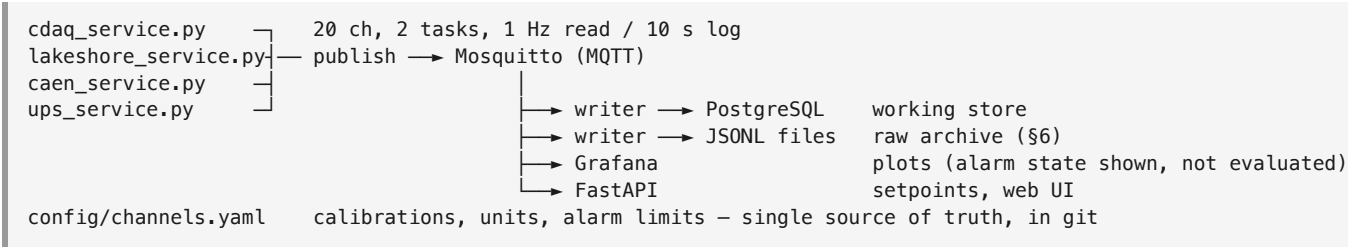
4. Still to recover

The documentation export renders only the active tab (`Input settings`). The tabs `High Voltage Supply`, `Monitor`, `Error and Alarm`, `Quick Temperature` and the phase diagrams were not exported, and they hold the remainder.

Missing	How to obtain
Alarm thresholds and responses	Screenshot of the <code>Error and Alarm</code> tab.
HV setpoints, ramp rates, trip and limit settings	Screenshot of the <code>High Voltage Supply</code> tab, plus the settings stored on the CAEN units themselves.
Physical position of each sensor, and what the 1xx / 2xx / 3xx series denote	Lab knowledge. Not recorded anywhere in software. The tags are now known; where they sit is not.
UPS model and connection	<code>check_ups.vi</code> , or simply looking at the unit.

Everything in this table except the sensor positions is an afternoon's work. The positions are the one item that must be verified empirically, tag by tag, once the read-only cDAQ service is logging raw values — warm one sensor by hand, see which value moves. The tag names are now known, but a tag that was already attached to the wrong channel in the LabVIEW system would be copied across silently. Physical verification is the only method that catches that.

5. Recommended architecture



One service per instrument, each an independent Windows service with its own reconnect loop, so a hung device cannot block the others. Channel definitions live in YAML, not in code:

```
- name: Pmain
  phys: 9207/ai5           # module alias, as LabVIEW uses
  kind: voltage
  offset: 0.0
  multiplier: 0.714
```

```

    unit: bar
    alarm: {low: 0.9, high: 1.8}

- name: tt301
  phys: 9216_1/ai0
  kind: rtd
  rtd: {type: PT3851, r0: 100.0, wiring: 3}
  unit: C
  legacy: TT301

```

Adding a sensor is then a wiring change plus one YAML entry — no code is touched. Every component is mainstream, individually replaceable, and plain text under version control.

6. Data logging

The CSV files on the lab PC are today's authoritative record — the MongoDB instance on the Nikhef server is temporary storage only. There is therefore no permanent archive at present: the whole history sits on one disk. **Copying it somewhere safe is worth doing today, independently of this project**, and the header file must be copied with it or the columns lose their meaning.

The present system writes a wide CSV table with the column names in a *separate* header file. This should not be carried over. A wide table places the meaning of column 17 outside the data file, so the data is unreadable without its companion, and — more dangerous — if a channel is ever added, removed or reordered, older files are silently misinterpreted. Nothing crashes; the numbers simply mean something other than what is assumed.

This is not hypothetical here. The LabVIEW front panel has a "Re-save headers" button, so the header file is overwriteable and may describe only the current layout. Any older file recorded under a different channel set would be read with its columns shifted. Counting the columns of every CSV in the history takes a minute and settles it.

Write long, not wide

One record per measurement, carrying its own name:

```

{"t":"2026-09-16T13:02:00.000Z","ch":"P101","v":1326.57,"raw":53.06,"u":"mbar"}
{"t":"2026-09-16T13:02:00.000Z","ch":"tt301","v":-92.30,"u":"C"}
{"t":"2026-09-16T13:02:00.000Z","ch":"HV_anode","v":3500.2,"u":"V","q":"ok"}

```

Consequences:

- Adding a channel does not touch existing data. No header migration, no column shift.
- A dropped channel is a missing record, not an empty column later mistaken for zero.
- A per-measurement quality flag ("q":"stale") fits naturally. In a wide CSV it cannot be expressed without breaking the format.
- A half-written final line after a crash is discarded; the rest of the file is intact.
- `grep P101 *.jsonl` works.

Sampling and logging rates

Sampling and logging are separate decisions. The hardware is read at **1 Hz**; a record is written every **10 s** as the mean of those ten samples — less noisy than any single reading, and six times less data than the present system.

Disk is not the constraint: even at 1 Hz the cost would be about 10 MB gzipped per day. The reasons to log less often are query speed, plot legibility and ease of analysis. At 10 s the system produces roughly 400 MB per year, so ten years of history fits in 4 GB.

The rate is uniform across all channels. Cryostat temperatures would be well served by 60 s, but identical timestamps let channels be compared without interpolation, and that simplicity outweighs the saving.

Flight recorder. The last 10 minutes at full 1 Hz are held in memory. When an alarm fires or a channel trips, the buffer is written to `data/events/`. This gives full resolution exactly around the moments that matter, without storing it

permanently — and it is what the present system lacks: after an incident there are only averaged values and no record of the approach to it.

Three layers, each with one job

Layer	Format	Purpose
Working store	PostgreSQL	What Grafana plots and what answers "pressure over the last month". A cache, not storage: rebuildable from the files.
Raw archive	JSON Lines, one file per day	Written by the ingest service independently of the database . If the database falls over or fills up, no data is lost. The files are the truth; the database is an index.
Long term	Parquet	The closed day is converted overnight and the JSONL gzipped. Compact, typed, loads straight into pandas, and still readable in ten years.

Two details that save a great deal later

Put a configuration hash in every file. First line of each daily file:

```
{"t":"2026-09-16T00:00:00Z","meta":{"config":"a3f91c2","version":"1.4.0","host":"xams-sc"}}
```

`channels.yaml` lives in git, so that hash states exactly which calibration was in force that day. This is the separate-header problem solved permanently rather than relocated.

Log the raw value alongside the scaled one, at least for the first year. If the multiplier of 25 on P101 later turns out to be wrong, the whole history can be rescaled. Without `raw`, that data is gone. At ~30 channels every 10 s the cost is roughly 12 MB per day uncompressed, around 1 MB gzipped — negligible on a 953 GB disk.

Safety — non-negotiable

The cDAQ is read-only, but the CAEN supplies and the Lake Shore heaters actuate. For those:

- **Interlocks belong in hardware, never in software.** If HV must trip on a vacuum or pressure excursion, wire it to the CAEN interlock input from a real gauge trip. Not a Python service reading MQTT and deciding.
- **Ramp rates, trip currents and over-voltage limits are configured on the CAEN units themselves.** Software verifies them at startup; it does not write them.
- **A restarted service must be a no-op on the hardware.** On connect it reads and publishes state. It does not restore setpoints, re-enable channels, or resume anything.
- **No write without a verified identity.** An HV service that cannot confirm the board serial number of the unit it is talking to must refuse to write at all.
- **Lake Shore heater output is real power into a cryogenic system.** The instrument's own setpoint limits and heater range are the authority; software sets a value within them and nothing more.

Note on the existing system. The current LabVIEW code does perform protective actions in software: a "Shut off heater if Alarm is active" switch and a `KILL VOLTAGE` button. This works in practice, but a software protection layer fails silently when the process hangs. The migration is the moment to decide explicitly: reproduce the behaviour as it stands, or move it into hardware. Reproducing it is defensible; doing so without noticing the choice is not.

7. Platform — Windows

The system stays on Windows, on the existing control PC. This reverses the earlier default of Linux, for reasons that only became clear once the inventory was known.

Consideration	Effect
NI-DAQmx and the cDAQ-9174	Decisive. Windows is NI's primary platform, so the compatibility risk around this discontinued chassis disappears entirely. On Linux it was the one item that could not be assumed.
LabVIEW remains installed	The old system stays startable within minutes as a rollback path. On a reinstalled Linux machine it would not.
No new hardware, no reinstall	The existing PC is more than adequate. Nothing needs procuring.
nidaqmx, pyserial, lakeshore, Grafana, Mosquitto, FastAPI	All run natively on Windows.
No systemd	Services run under NSSM (which wraps an executable as a Windows service) or Task Scheduler. Works well, but automatic restart and log rotation must be configured explicitly rather than in a four-line unit file.
No udev	Device identification moves into Python via <code>serial.tools.list_ports</code> plus the CAEN <code>BDSNUM</code> check. This is better practice regardless of platform: it makes the operating system a deployment detail rather than an architectural dependency.
Database choice	PostgreSQL, which has a native Windows installer and a built-in Grafana datasource. InfluxDB was the obvious default but its Windows support is uneven. TimescaleDB is a PostgreSQL extension, not an alternative, and is not needed at this volume.

Do not run the hardware services under WSL2. USB passthrough to WSL2 does not work with NI-DAQmx. Python runs natively on Windows for anything that touches an instrument; Docker Desktop is fine for the infrastructure layer, which does not.

This also settles §9 more firmly: EPICS effectively requires Linux, and the platform decision now runs the other way.

8. Running the services

Each service is a script that loops indefinitely:

```
while True:
    values = read_hardware()
    publish(values)
    time.sleep(2)
```

It is not started by hand. NSSM wraps it as a Windows service:

```
nssm install xams-cdaq "C:\xams\venv\Scripts\python.exe" "C:\xams\cdaq_service.py"
nssm set xams-cdaq AppDirectory C:\xams
nssm set xams-cdaq AppStdout C:\xams\logs\cdaq.log
nssm set xams-cdaq AppStderr C:\xams\logs\cdaq.err
nssm set xams-cdaq AppExit Default Restart
nssm set xams-cdaq AppRestartDelay 5000
```

The service then restarts after a crash, runs without a logged-in user, and appears in `services.msc` like any other. This is already an improvement on the present situation, where someone must open LabVIEW and press Run — after a reboot the slow control stands still until a person notices.

Automatic start stays off while LabVIEW is the fallback

Because every device admits only one process (§11), automatic start during the transition is not merely unnecessary — it is dangerous.

The failure mode: the PC reboots overnight after an update, the Python service starts automatically and claims the cDAQ chassis and the COM ports. The next morning LabVIEW cannot start, because the hardware is taken. **The fallback has been locked out by the system that is still being debugged.**

Automatic start is therefore enabled only in the final phase, after LabVIEW is retired.

Phase	How it runs	Why
1. Debug	No service at all. <code>python cdaq_service.py</code> from a terminal.	Tracebacks appear on screen rather than in a log file. The process should die on an error, not be silently restarted by NSSM while the fault goes unnoticed.
2. Trial running	Installed as a service with <code>Start SERVICE_DEMAND_START</code> ; started and stopped by hand.	Restarts after a crash, but does not start at boot. After a reboot the hardware is free and LabVIEW can start normally.
3. Production	<code>Start SERVICE_AUTO_START</code>	Comes back by itself after a power event — relevant, given that there is a UPS. Enabled only once LabVIEW is no longer the fallback.

The difference between phase 2 and phase 3 is one line per service.

Two things to build in from the start

- **A clear message when a device is already in use.** Starting a service while LabVIEW holds the hardware should produce `FATAL: cDAQ1 in use by another process (LabVIEW running?)`. Refusing to start. — not a ten-digit DAQmx error code. This happens often during the transition.
- **A lock file or named mutex**, so a second instance of the same service cannot start. Duplicate instances produce duplicate data, and the cause is usually looked for in the hardware first.

One switch

```
xams-ctl stop    # stop all services, release the hardware for LabVIEW
xams-ctl start   # start all services
xams-ctl status  # what is running, and how old each heartbeat is
```

Switching four services on and off individually goes wrong eventually, usually in a hurry. A single command that always does it in the right order is the most-used tool of the transition period.

"Running" is not "working"

A service reported as Running may have been doing nothing useful for three days: a hung serial port, an unhandled exception inside a loop, an unplugged device. Windows sees a live process and is satisfied.

Each service therefore publishes a heartbeat:

```
xams/status/cdaq/heartbeat 2026-09-16T13:02:00Z
```

A small monitor raises an alarm when a heartbeat is older than a minute. The same principle applies per channel: a value that is no longer being refreshed is published with `"q":"stale"` rather than left standing at its last reading. A frozen value that looks healthy is more dangerous than a missing one.

9. EPICS — considered and set aside

EPICS is the standard control system at accelerators, light sources and large detector experiments, and was evaluated properly. An IOC (Input/Output Controller) serves named channels — PVs — on the network, declared as records in text files, with scan rate, engineering units and alarm limits as record fields rather than code:

```
record(ai, "XAMS:PRESSURE:MAIN") {
    field(DTYP, "stream")
    field(SCAN, "1 second")
    field(EGU, "bar")
    field(HIGH, "1.8")    field(HSV, "MINOR")
    field(HIHI, "2.0")   field(HHSV, "MAJOR")
}
```

In its favour	Against, at this scale
Declarative records: alarm limits, units, deadbands and simulation mode come free and standardised	Four instruments and ~50 channels do not justify the machinery. More time would go to <code>epics-base</code> and Makefiles than to the system itself
StreamDevice handles the Lake Shore 335 and the CAEN ASCII protocol elegantly; EPICS support for both exists in the community	No maintained EPICS driver for CompactDAQ. The cDAQ backbone would need a Python soft IOC (<code>pcaspy</code> / <code>p4p</code>) regardless — the Python gets written either way
Flat global PV namespace; genuine per-IOC process isolation	Peripheral infrastructure is heavy: the Phoebus alarm server needs Kafka, Archiver Appliance needs Java, Tomcat and MySQL
Skills and drivers transfer to and from larger experiments	Linux effectively required; steep learning curve for a new student
Proven over decades of continuous operation	Bus-factor risk is relocated, not solved. A one-person EPICS installation nobody else can boot or debug is no more maintainable than the present LabVIEW project

Reconsider EPICS if a second maintainer with controls interest is available, Nikhef controls-group support can be drawn on, or XAMS is expected to grow substantially or feed a larger controls effort. Absent those, it is the wrong tool for this system.

10. Effort

Component	Estimate
cDAQ service — 2 tasks, 20 connected channels, read only	2–3 days
Lake Shore 335 service	1 day
CAEN HV service — ASCII over serial, monitoring first, control later	2–3 days
UPS service	0,5 day
MQTT, PostgreSQL, Grafana, dashboards	2–3 days
Logging: JSONL writer, nightly Parquet conversion	1 day
Alarms — SMS, email, thresholds — documentation	2–3 days
Service wrapping (NSSM), <code>xams-ctl</code> , heartbeat monitor	1 day
Commissioning, empirical channel identification, and fixing what reality objects to	The remainder, and the part that genuinely takes time

11. Sequence

- Collect the remaining configuration** (§4): tab screenshots, a log file, UPS details.

2. **Set up the development environment on the control PC** — Python, git, the repository checked out locally. The machine stays as it is; nothing is reinstalled.
3. **Implement hardware-ID resolution and the serial-number identity check** on the HV services, before they are ever allowed to write.
4. **Build the cDAQ service.** Read-only, so it cannot disturb the detector. Test it in short windows with LabVIEW stopped, logging raw values under their channel numbers.
5. **Verify each tag against its channel empirically** and record the physical locations in `channels.yaml`. Only then apply scaling and alarms.
6. **Add Lake Shore, CAEN and UPS monitoring** — reading only, no writes.
7. **Validate against the existing CSV history.** Same sensors, different days: do values and noise levels agree? This catches scaling errors without anything needing to run simultaneously.
8. **Only then migrate control**, one device at a time, having decided what belongs in hardware. This is the sole step requiring real care.

Why there is no parallel-running phase

An earlier draft of this plan proposed running the new system alongside LabVIEW for a month. That is not possible with this hardware, and the reason is worth stating so it is not proposed again.

- **CompactDAQ permits one analog-input task per chassis.** While LabVIEW holds the tasks, the Python service cannot reserve the chassis; it simply fails to start.
- **A COM port admits one process.** The same applies to both CAEN supplies and the Lake Shore.
- **Everything is USB**, so it is bound to a single host. A second machine does not help.

The migration is therefore a short, planned cut-over per device, with LabVIEW left installed and startable within minutes as the rollback path. That is a more honest safety net than simultaneity, and in practice a better one: it is exercised by actually using it, rather than assumed to work.

12. Open questions

- Whether the two DT1470ET units expose unique USB serial numbers
- Whether the 9207 current channels `ai8:15` are genuinely unused
- What `anode_timing.vi` and `DAISY_polarity_signs.vi` do, and whether that behaviour must be reproduced
- Whether heater shut-off and HV kill move into hardware or stay in software
- A spare or successor path for the cDAQ-9174, which is a discontinued model — modules carry over unchanged to a cDAQ-9179 or 9189
- Who, besides the primary author, will be able to maintain the system